LAB: Grayscale Image Segmentation - Gear

Date: 2026-03-24 **Author:** lee jeayong(22000561)

Introduction

1. Objective

The goal of this lab is to develop a machine vision system that can inspect defective plastic gears. We are asked to segment the target objects from the background and determine the number of defective teeth and quality by applying thresholding, morphology, and contour analysis.

The system will calculate the following:

- Number of defective teeth
- Diameter of the gear
- Quality Inspection (Pass or Fail)

2. Preparation

Software Installation

- OpenCV, C++

Dataset

- `Lab_GrayScale_Gears.zip` (Gear1.jpg, Gear2.jpg, Gear3.jpg, Gear4.jpg)

Algorithm

1. Overview

The algorithm follows these main steps:

1. **Image Load & Grayscale Conversion:** Read the image and convert it to grayscale.
2. **Edge Extraction:** Apply a Laplacian filter to extract edges and threshold it to a binary image.
3. **Solid Gear Masking:** Find boundaries of the object and fill the contours to create a solid mask of the entire gear.
4. **Inner Body Separation:** Apply Distance Transform to the solid mask to find the center and the root radius of the gear. Create a mask for the inner circular body.
5. **Teeth Extraction:** Subtract the inner body mask from the solid gear mask to isolate the teeth. Apply morphological opening to denoise.
6. **Defect Detection:** Find contours of the isolated teeth, calculate their areas, and compute the average area. A tooth is considered defective if its area is less than 90% or more than 110% of the average area (i.e., broken or missing).

2. Procedure

Edge Detection

Since the gears share similar intensity values with some parts of the background, detecting edges directly is more robust than simple thresholding. Thus, a `Laplacian` filter (kernel size 3) is applied to the grayscale image. The result is binarized using a threshold value of 30.

Contour and Solid Mask

`findContours` is applied to the binarized edges to find the external boundaries. The contours are then filled using `drawContours` with `FILLED` mode to generate a solid mask covering the entire gear.

Distance Transform

To isolate the teeth from the gear body, we find the inscribed circle of the gear's inner body. `distanceTransform` is applied to the solid mask. Using `minMaxLoc`, the pixel with the maximum distance to the background is found, which corresponds to the gear's center, and its value corresponds to the inner radius. This allows building a circular mask representing the body.

Defective Teeth Determination

The inner body mask is subtracted from the solid gear mask, leaving only the gear teeth. `morphologyEx` (MORPH_OPEN) removes remaining noise. Consequent `findContours` groups each tooth individually. The areas of all valid teeth contours (>15.0) are measured using `contourArea`, and an average area is calculated. If a contour's area falls below 90% or exceeds 110% of the average area, it is flagged as a defective tooth.

Result and Discussion

1. Final Result

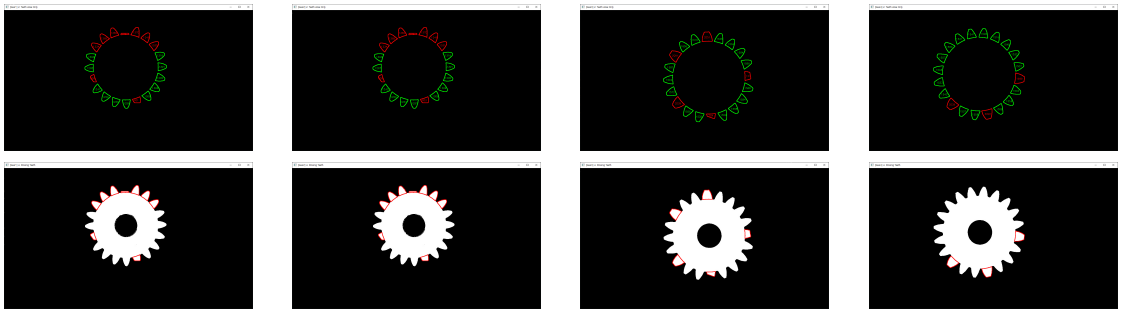
For each gear image, the program successfully outputs:

- Original image
- Teeth Area Only image (showing segmented teeth and their areas)
- Teeth Outlines Only image (showing normal and defective teeth outlines)
- Missing Teeth image (original image highlighting the defects)

In the terminal, the algorithm prints the number of detected teeth, the average area of the teeth, and the broken/defective teeth count.

Analysis Results

	Sample #1	Sample #2	Sample #3	Sample #4

	Sample #1	Sample #2	Sample #3	Sample #4
Output Images				
Teeth numbers	20	20	20	20
Avg. Teeth Area	1204.2	1204.7	1586.6	1611.5
Defective Teeth	9	9	5	3
Quality	FAIL	FAIL	FAIL	FAIL

2. Discussion

The proposed algorithm robustly segments the plastic gear using a Laplacian filter and Distance Transform, making it independent of exact lighting conditions that might hinder simple global thresholding. Using the average area of the teeth for comparison effectively adapts the defect threshold for different gear sizes or distances from the camera.

Conclusion

The machine vision system successfully detected defective teeth on the given plastic gear images. By utilizing edge detection, morphological processing, and distance transformation, the algorithm successfully separated the gear teeth from the body and identified the defective ones based on their individual area sizes.

Appendix

main.cpp

```
#include <opencv2/opencv.hpp>
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>

using namespace cv;
using namespace std;
```

```
int main() {
    vector<string> file_names = {"Gear1.jpg", "Gear2.jpg", "Gear3.jpg",
"Gear4.jpg"};

    for (int idx = 0; idx < file_names.size(); idx++) {
        string file_name = file_names[idx];
        string prefix = "[Gear" + to_string(idx + 1) + "] ";

        // 1. 이미지 읽기
        Mat img_color = imread(file_name, IMREAD_COLOR);
        if (img_color.empty()) {
            cerr << prefix << "이미지를 불러올 수 없습니다." << endl;
            continue; // 다음 이미지로 넘어감
        }

        Mat img_gray;
        cvtColor(img_color, img_gray, COLOR_BGR2GRAY);

        // ------------------------------------------------------------
        // 2. Laplacian 필터 및 이진화
        // ------------------------------------------------------------
        Mat img_laplacian, edges_binary;
        Laplacian(img_gray, img_laplacian, CV_16S, 3);
        convertScaleAbs(img_laplacian, img_laplacian);
        threshold(img_laplacian, edges_binary, 30, 255, THRESH_BINARY);

        // ------------------------------------------------------------
        // 3. 솔리드(Solid) 마스크 생성
        // ------------------------------------------------------------
        vector<vector<Point>> edge_contours;
        findContours(edges_binary, edge_contours, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE);

        Mat mask_solid = Mat::zeros(img_gray.size(), CV_8U);
        drawContours(mask_solid, edge_contours, -1, Scalar(255), FILLED);

        // ------------------------------------------------------------
        // 4. 거리 변환으로 뿌리원 몸체 분리
        // ------------------------------------------------------------
        Mat dist;
        distanceTransform(mask_solid, dist, DIST_L2, 5);

        double minVal, maxVal;
        Point minLoc, maxLoc;
        minMaxLoc(dist, &minVal, &maxVal, &minLoc, &maxLoc);

        Mat mask_body = Mat::zeros(img_gray.size(), CV_8U);
        int root_radius = cvRound(maxVal) + 1;
        circle(mask_body, maxLoc, root_radius, Scalar(255), FILLED);

        Mat mask_teeth;
        subtract(mask_solid, mask_body, mask_teeth);

        Mat small_kernel = getStructuringElement(MORPH_RECT, Size(3, 3));
```

```
        morphologyEx(mask_teeth, mask_teeth, MORPH_OPEN, small_kernel);

        // ---------------------------------------------------------
        // 5. 이빨 면적 계산 및 불량 판정
        // ---------------------------------------------------------
        vector<vector<Point>> all_teeth_contours;
        findContours(mask_teeth, all_teeth_contours, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE);

        vector<vector<Point>> contours;
        for (size_t i = 0; i < all_teeth_contours.size(); i++) {
            if (contourArea(all_teeth_contours[i]) > 15.0) {
                contours.push_back(all_teeth_contours[i]);
            }
        }

        double total_area = 0.0;
        for (size_t i = 0; i < contours.size(); i++) {
            total_area += contourArea(contours[i]);
        }
        double avg_area = contours.empty() ? 0.0 : total_area / contours.size();

        // 정상 이빨 기준 (평균의 90% ~ 110%)
        double min_normal_area = avg_area * 0.9;
        double max_normal_area = avg_area * 1.1;

        // ---------------------------------------------------------
        // 6. 결과 렌더링용 캔버스 생성
        // ---------------------------------------------------------
        Mat img_res2 = Mat::zeros(img_color.size(), CV_8UC3); // 까만 배경 + 이빨선
+ 텍스트
        Mat img_res3 = Mat::zeros(img_color.size(), CV_8UC3); // 까만 배경 + 이빨선
(텍스트 없음)
        Mat img_res4 = img_color.clone(); // 4번째는 원본 배경 위에 불량 표기

        int defect_count = 0;

        for (size_t i = 0; i < contours.size(); i++) {
            double area = contourArea(contours[i]);

            bool is_defect = (area < min_normal_area || max_normal_area < area);

            if (is_defect) defect_count++;

            Scalar color = is_defect ? Scalar(0, 0, 255) : Scalar(0, 255, 0);

            drawContours(img_res2, contours, (int)i, color, 2);
            drawContours(img_res3, contours, (int)i, color, 2);

            if (is_defect) {
                drawContours(img_res4, contours, (int)i, Scalar(0, 0, 255), 2);
            }

            Moments M = moments(contours[i]);
```

```
            if (M.m00 != 0) {
                int cX = int(M.m10 / M.m00);
                int cY = int(M.m01 / M.m00);
                putText(img_res2, to_string((int)area), Point(cX - 15, cY + 5),
FONT_HERSHEY_SIMPLEX, 0.4, color, 1);
            }
        }

        cout << "\n\n[" << file_name << "] 분석 결과" << endl;
        cout << "=========================================" << endl;
        cout << "| " << left << setw(10) << "이빨 개수"
             << "| " << setw(12) << "평균 면적"
             << "| " << setw(12) << "불량 개수" << " |" << endl;
        cout << "-----------------------------------------" << endl;
        cout << "| " << left << setw(10) << contours.size()
             << "| " << setw(12) << fixed << setprecision(1) << avg_area
             << "| " << setw(12) << defect_count << " |" << endl;
        cout << "=========================================" << endl;

        imshow(prefix + "1. Original", img_color);
        imshow(prefix + "2. Teeth Area Only", img_res2);
        imshow(prefix + "3. Teeth Outlines Only", img_res3);
        imshow(prefix + "4. Missing Teeth", img_res4);
    }

    cout << "\n>> 모든 분석이 완료되었습니다. 창에서 아무 키나 누르면 종료됩니다..."
<< endl;
    waitKey(0);

    return 0;
}
```